

Progetto di Laboratorio di Sistemi Operativi

Forza 4 in rete: server C multi-client e client Java

[Nome Cognome]

[Nome Cognome] (se presente in coppia)

Matricola: [Matricola]

Corso di Laurea in Informatica

Anno Accademico: [Anno Accademico]

1. Architettura generale

Il sistema e' composto da un server scritto in C, eseguito in un container Docker, e da un numero arbitrario di client scritti in Java (Swing), ciascuno eseguito come processo indipendente. Client e server comunicano tramite socket TCP (nessun websocket o libreria di rete di livello superiore), scambiandosi messaggi testuali in un formato ispirato a JSON, terminati da un carattere di a-capo.

La scelta di un server multi-thread (un thread per connessione, creato con *pthread_create* e reso indipendente con *pthread_detach*) invece che a processi figli o basato su *select()* su un unico thread e' motivata dalla necessita' di condividere in modo naturale strutture dati in memoria (l'elenco delle partite e dei giocatori) tra tutte le connessioni attive, senza dover ricorrere a memoria condivisa esplicita o IPC tra processi separati. Il rovescio della medaglia e' la necessita' di sincronizzare esplicitamente gli accessi concorrenti, discussa nella sezione 3.

Il client e' organizzato secondo uno schema MVC semplificato: **GameModel** mantiene lo stato locale della partita corrente (board, turno, id/nome dei due giocatori, stato della partita), **GameController** traduce gli eventi dell'interfaccia in messaggi di rete e i messaggi ricevuti in aggiornamenti del model e della vista, mentre il package **view** contiene i pannelli Swing (lobby, schermata di attesa per una richiesta di partecipazione, board di gioco).

2. Protocollo di comunicazione applicativo

Ogni messaggio scambiato tra client e server e' un oggetto con quattro campi: *type* (REQUEST, RESPONSE o NOTIFICATION), *action* (il nome dell'operazione, es. "create_match"), *msg_id* (identificativo generato dal client per correlare richiesta e risposta) e *payload*, un sotto-oggetto il cui contenuto dipende dall'azione. Le NOTIFICATION sono messaggi non richiesti esplicitamente, usati dal server per informare un client di eventi generati da un altro giocatore (una mossa, l'inizio di una partita, l'abbandono dell'avversario, ecc.): senza questo canale, un client non avrebbe modo di sapere che l'avversario ha giocato se non interrogando continuamente il server (polling), soluzione piu' dispendiosa e meno reattiva.

Lato server, il parsing dei messaggi in ingresso e' realizzato con semplici chiamate a *sscanf* su pattern fissi, invece di una libreria JSON completa. La scelta e' stata deliberata: il progetto e' vincolato all'uso del C puro senza dipendenze esterne pesanti, e il numero di campi per ciascuna azione e' piccolo e noto a priori, rendendo un parser "su misura" sufficiente e piu' semplice da mantenere rispetto all'integrazione di una libreria come cJSON. Il limite di questo approccio e' che un parser basato su pattern fissi non gestisce correttamente caratteri di escape JSON (es. virgolette all'interno di una stringa): questo e' stato mitigato sanificando lato client i campi liberi (in particolare lo username scelto dall'utente) prima dell'invio, rimuovendo i caratteri che romperebbero il parsing, e aggiungendo comunque una funzione di escape lato server per l'output (in modo che un'eventuale stringa problematica non comprometta comunque i messaggi generati dal server verso gli altri client).

3. Concorrenza e sincronizzazione lato server

Le strutture dati condivise tra i thread sono due: l'array delle partite (*matches*) e l'array dei giocatori connessi (*players*), ciascuno protetto da un proprio mutex globale (rispettivamente *matches_mutex* e *players_mutex*). A questi si aggiunge un mutex per connessione (*send_mutex*, uno per ogni giocatore), necessario perche' un thread puo' dover scrivere sul socket di un giocatore diverso da se' stesso (per inoltrargli una NOTIFICATION): senza questo mutex, due thread potrebbero scrivere contemporaneamente sullo stesso socket, corrompendo il messaggio ricevuto dal client.

Per evitare deadlock, e' stato adottato e rispettato in tutto il codice un ordine di acquisizione fisso dei lock: prima *matches_mutex*, poi (se necessario) *players_mutex*, infine il *send_mutex* del singolo giocatore. La funzione che notifica tutti i client connessi tranne i due partecipanti a una partita (usata per informarli che una partita e' iniziata o si e' conclusa) rispetta questa disciplina raccogliendo prima, sotto *players_mutex*, un semplice elenco di identificativi dei destinatari, per poi rilasciare il lock e inviare i messaggi uno alla volta: in questo modo non si tiene mai *players_mutex* mentre si acquisisce a cascata un *send_mutex*, evitando un possibile deadlock con thread che stessero facendo il percorso inverso.

4. Macchina a stati della partita

Ogni partita e' modellata come una macchina a stati finiti con cinque stati: **WAITING_FOR_OPPONENT** (creata, nessuna richiesta di adesione in corso), **JOIN_PENDING** (c'e' una richiesta di partecipazione che il creatore sta valutando), **ACTIVE** (due giocatori assegnati, partita in corso), **FINISHED** (conclusa con vittoria, sconfitta o pareggio) e **ABANDONED** (chiusa senza essere arrivata a conclusione naturale: rifiutata, lasciata da un giocatore, o mai avviata). La distinzione tra **WAITING_FOR_OPPONENT** e **JOIN_PENDING** e' stata introdotta per poter distinguere, quando arriva una nuova richiesta di partecipazione, se diventare subito la richiesta "attiva" (nessuno la sta gia' valutando) oppure se metterla in coda (il creatore sta gia' decidendo su un'altra richiesta): senza questo stato intermedio sarebbe stato necessario un campo booleano separato per rappresentare la stessa informazione, meno leggibile nel resto del codice che gia' ragiona per stati.

Lo stato **ABANDONED** e' distinto da **FINISHED** perche' una partita puo' terminare "innaturalmente" in diversi momenti (nessun avversario si e' mai presentato, un giocatore ha abbandonato a meta' partita, una richiesta di adesione e' stata chiusa senza risposta): trattarle tutte come **FINISHED** avrebbe reso ambigua, in fase di rigioco, la domanda "chi ha vinto?" per casi in cui in realta' non si e' mai giocato.

5. Richieste di partecipazione: coda ed esclusione

La traccia richiede che il creatore possa accettare o rifiutare esplicitamente ogni richiesta di adesione, e non che l'adesione sia automatica. Questo introduce un problema di concorrenza specifico: se piu' client richiedono di partecipare alla stessa partita mentre il creatore sta ancora valutando la prima richiesta, cosa succede alle altre? La soluzione adottata e' una coda FIFO (implementata come semplice array a dimensione fissa con scorrimento) di richieste in attesa: la prima richiesta diventa quella "in decisione" e genera il popup sul client del creatore, le successive vengono accodate e informate di essere in coda. Quando la richiesta in decisione viene rifiutata (o chi l'ha fatta la annulla di propria iniziativa), la prossima in coda viene automaticamente promossa e proposta al creatore, mantenendo l'ordine di arrivo.

Un giocatore rifiutato viene inserito in una lista di "rifiutati" associata alla singola partita, per impedirgli di ripresentare la stessa richiesta sulla stessa partita (mentre resta libero di provare altre partite). Questa lista viene azzerata quando la partita viene riaperta come nuova sessione da un vincitore (si veda la sezione 7), perche' a quel punto e' a tutti gli effetti un'occasione di gioco nuova.

Uscite e disconnessioni durante una richiesta

Se chi ha fatto la richiesta la annulla, oppure si disconnette, mentre il creatore ha gia' il popup di conferma aperto, il server lo sa (aggiorna subito lo stato interno) ma il client del creatore non puo' "richiudere" un popup di sistema gia' visualizzato: se il creatore risponde comunque, il server riconosce che la richiesta non esiste piu' e gli comunica esplicitamente chi l'ha ritirata, invece di un

generico errore "nessuna richiesta da accettare", per evitare che il messaggio sembri un malfunzionamento.

6. Visibilita' delle partite e notifiche broadcast

La lista delle partite restituita a un client (azione *list_matches*) include tutte le partite negli stati `WAITING_FOR_OPPONENT`, `JOIN_PENDING` e `ACTIVE`, mai quelle `FINISHED` o `ABANDONED`. Le prime due sono mostrate come disponibili (con etichetta diversa a seconda che ci sia gia' una richiesta in decisione), la terza e' mostrata come "in corso", visibile ma non selezionabile per una richiesta di adesione: questo rispecchia il requisito per cui i client non coinvolti devono sapere che una partita esiste ed e' in corso, senza poter interferire con essa.

Oltre all'aggiornamento passivo (ogni client richiede periodicamente la lista aggiornata), il server invia una notifica broadcast, a tutti i client connessi tranne i due partecipanti, nei due momenti in cui lo stato di visibilita' di una partita cambia: quando una richiesta viene accettata (la partita passa da disponibile a "in corso") e quando la partita si conclude o viene abbandonata (torna a non essere piu' visibile). Senza questo meccanismo, gli altri client si accorgerebbero del cambiamento solo al successivo aggiornamento periodico della lista, con un ritardo percepibile.

7. Fine partita e rigioco

Alla conclusione di una partita il server memorizza, insieme allo stato `FINISHED`, anche l'identificativo del vincitore (o un valore convenzionale per il pareggio). Questa informazione persistente e' cio' che permette, in un momento successivo e del tutto asincrono, di distinguere correttamente due comportamenti diversi richiesti dalla traccia per il rigioco (opzionale):

- in caso di vittoria o sconfitta, solo il vincitore puo' decidere di aprire una nuova sessione sulla stessa partita, diventandone proprietario (anche se non lo era gia', cioe' anche se in origine era stato lui a unirsi e non a creare la partita). Il perdente non ha alcuna scelta: viene riportato automaticamente in lobby, e questa sua uscita e' trattata come un evento silenzioso che non deve interferire con l'eventuale decisione, ancora pendente, del vincitore;
- in caso di pareggio, la traccia richiede che la decisione sia congiunta: il primo giocatore che accetta di rigiocare viene messo in attesa (il suo voto viene solo registrato), e la partita riparte, con la stessa coppia di giocatori, solo quando anche il secondo accetta. Se uno dei due rifiuta, l'altro (se stava aspettando) viene avvisato invece di restare in attesa indefinitamente.

Quando il vincitore apre una nuova sessione, la partita torna nello stato `WAITING_FOR_OPPONENT` con il vincitore come nuovo proprietario/giocatore 1: non e' quindi detto che il prossimo avversario sia lo stesso di prima, esattamente come una partita creata da zero, in coerenza con la richiesta della traccia di "invitare nuovamente gli altri giocatori a partecipare".

Una conseguenza di questo modello e' che un giocatore che ha appena perso deve essere considerato immediatamente libero di creare o richiedere un'altra partita, anche se la partita appena conclusa non e' stata ancora chiusa definitivamente dal vincitore: il controllo lato server che impedisce di partecipare a piu' partite contemporaneamente tiene conto di questa asimmetria, distinguendo esplicitamente il ruolo di vincitore e perdente invece di trattarli allo stesso modo solo perche' entrambi compaiono nella stessa partita conclusa.

8. Gestione di abbandoni e disconnessioni

Il comportamento alla chiusura di una partita da parte di un giocatore dipende dallo stato in cui si trova, non e' un'unica azione valida sempre allo stesso modo: se la partita e' attiva, uscire equivale ad abbandonare, e l'avversario vince a tavolino; se la partita e' ancora in attesa di un avversario (o di una decisione su una richiesta), viene semplicemente chiusa, avvisando pero' chi avesse una richiesta di adesione in corso o in coda su di essa; se la partita e' gia' conclusa, si applica la distinzione vincitore/perdente descritta nella sezione precedente.

La disconnessione improvvisa di un client (chiusura del processo, rete interrotta) e' trattata allo stesso modo di un'uscita esplicita, cercando tra tutte le partite quella a cui il giocatore disconnesso partecipava (come giocatore, come richiedente in decisione o come richiedente in coda) e applicando la stessa logica di chiusura/abbandono, cosi' che l'eventuale avversario o gli altri richiedenti non restino mai bloccati in attesa di un evento che non arrivera' piu'.

9. Architettura del client Java

Il client usa Swing con un'unica finestra (*JFrame*) il cui contenuto viene sostituito a seconda del contesto (lobby, schermata di attesa per una richiesta di adesione, board di gioco), invece di aprire finestre multiple: questo evita di dover gestire lo stato di piu' finestre indipendenti e riflette il fatto che, in ogni momento, un client puo' trovarsi in una sola di queste situazioni.

Tutta la gestione dei messaggi di rete in arrivo viene fatta rientrare, tramite *SwingUtilities.invokeLater*, sull'Event Dispatch Thread: i socket vengono letti su un thread dedicato, ma ogni aggiornamento del model e della vista avviene sempre sull'EDT, come richiesto dal modello di programmazione di Swing (i componenti Swing non sono thread-safe). Alcuni elementi dell'interfaccia (l'avviso temporaneo di 2 secondi quando un avversario si unisce, l'aggiornamento automatico della lobby ogni pochi secondi) sono realizzati con *javax.swing.Timer*, che gia' esegue i propri callback sull'EDT, evitando cosi' ulteriori sincronizzazioni manuali.

10. Limiti noti e possibili sviluppi futuri

- Lo stato del server e' interamente in memoria: un riavvio del server comporta la perdita di tutte le partite e le registrazioni in corso. Una possibile estensione e' la persistenza su file o database.
- Il parser del protocollo, basato su pattern fissi, non e' un parser JSON completo: non gestisce correttamente caratteri di escape complessi. E' stato mitigato sanificando lo username in input, ma un client scritto ad-hoc per "rompere" il protocollo potrebbe in teoria costruire payload malformati.
- Non e' previsto un timeout per una richiesta di partecipazione lasciata in sospeso a tempo indeterminato: se il creatore non risponde mai, chi ha fatto richiesta resta in attesa finche' non la annulla lui stesso. Un timeout automatico sarebbe un'estensione naturale.
- Il numero massimo di partite e di giocatori connessi contemporaneamente e' limitato da costanti fissate a tempo di compilazione (100 e 100): sufficiente per l'uso didattico previsto dal progetto, ma da rivedere per un utilizzo con molti piu' utenti.